

Library Database

Sean Bush

Database design can significantly improve or hinder an organization's ability to gather insights from their data. In this project, I will be showing how one might go about designing a database for use in a city library. I will take a set of data that is currently not well-suited for long-term data storage and design a better, more-normalized data schema. Then I will implement this schema in a new PostgreSQL database, import the data from the old schema into the new database, and illustrate how to query the database to answer common organizational questions. Lastly, I will show how indexes can be used to optimize common queries.

Dataset Generation

This project was conceived several years ago, when I was first learning to use PostgreSQL for database management. I obtained the dataset originally from some third-party source like Kaggle and used it to practice what I had recently learned about database management, with no intention to publish it. When I returned recently to complete the project and publish it as a part of my professional portfolio, I went searching for the source of the dataset to ensure that it was released with a license that permitted free usage, but I could no longer ascertain the source of the dataset. As such, I determined that I had to create a new dataset from scratch using the Python library Faker in order to assure that I would have the right to use and publish the dataset (Faker contributors, n.d.).

The code used to generate the dataset is accessible from this project's [GitHub Repository](#) in the "data" directory. The script called "generate.py" performs the data generation, and the Dockerfile located in the directory contains the blueprint for creating a Docker container with all of the necessary dependencies (Docker, Inc., n.d.). In order to access the generated dataset from outside the container, the container's working directory needs to be mounted to a directory on the user's file system by adding the following flags to the docker run command: "docker run -v ./:/app generate_script" (Docker, Inc. n.d.). A batch script is included in the repository which will build and run the container automatically assuming the user is on a Windows machine and has the Docker engine running.

The generated dataset has the following schema. There is a single table with ten columns: BookID, Title, Author, Genre, BranchID, BranchName, BorrowerID, BorrowerName, BorrowDate, and ReturnDate.

A	B	C	D	E	F	G	H	I	J
BookID	Title	Author	Genre	BranchID	BranchName	BorrowerID	BorrowerName	BorrowDate	ReturnDate
1	Serious inside else memory if.	Marvin Watson	History	1	Branch-1	28	Adrienne Griffin	2024-11-10	
2	Whose group through despite.	Teresa Mitchell	History	3	Branch-3	58	Stacy Foster	2024-03-04	2025-11-09
3	Example sense peace.	William Stevens	Classic Fiction	3	Branch-3	56	Ethan Ward	2024-02-23	
4	Would mouth relate own.	Stephanie Gonzalez	Horror	1	Branch-1	19	Timothy Walton	2024-01-19	2025-07-05
5	Financial role together.	Michael Rivera	Biography	5	Branch-5	46	Lori Gibbs	2024-03-29	
6	Line beyond its particularly tree.	John McClain	History	2	Branch-2	40	Amy Moses	2024-12-22	
7	Local tend employee source nature add.	Sherri Richardson	Horror	2	Branch-2	23	Gary Johnson	2024-02-05	2025-11-15
8	Report size personal partner stock.	Damon Henry	Biography	2	Branch-2	84	Ashley Hill	2024-08-01	2025-06-16
9	During foot that course nothing.	Brandon Stone	Non-fiction	4	Branch-4	43	Rebecca Williams	2024-03-28	
10	Sound central myself before.	Kristen Austin	Mystery	1	Branch-1	94	Robert Hubbard	2024-02-12	
11	Themselves your majority feeling fact.	Jeffrey Stephens	Sci-Fi/Fantasy	4	Branch-4	96	Michael Savage	2024-01-24	

Each record in this table represents a single instance of a book being borrowed from the library. The BookID column uniquely identifies each record. The next three columns (Title, Author, and Genre) describe the book being borrowed in this record. The BranchID and BranchName columns describe the library branch from which the book was borrowed. The BorrowerID and BorrowerName columns describe the library member who borrowed the book. Lastly, the BorrowDate and ReturnDate columns describe when the book was borrowed and returned respectively.

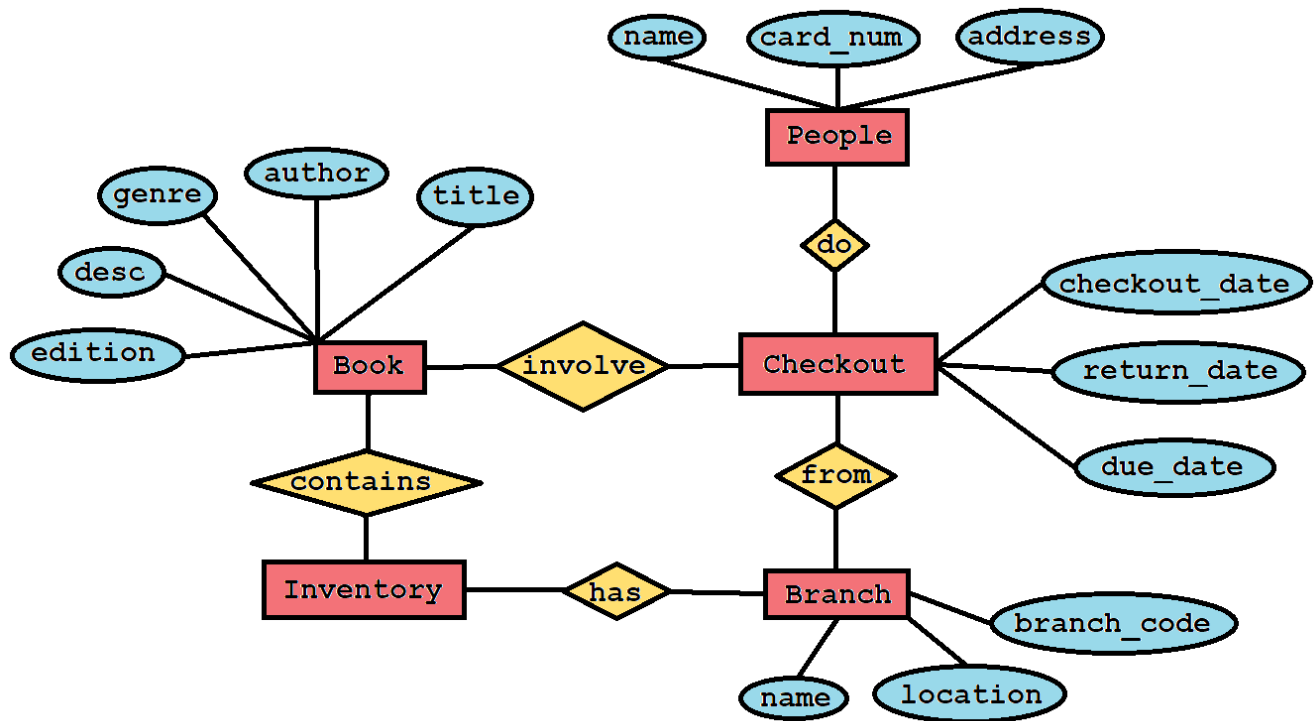
With no supplied data dictionary describing the columns in detail, determining the precise meaning of each column requires some interpretation. For example, the BookID column does not appear to be identifying the book itself but rather each individual instance of someone borrowing a book. Admittedly, it could be interpreted either way, since the BookID does uniquely identify each Title, Author combination, but the order of BookID implies that it is meant to be a primary key for this table specifically, and it was taken as such. Other interpretive leaps that I made while considering the dataset were assuming that ReturnDate column refers to the date a book was actually returned rather than it being the due date for that borrow. Accordingly, I also assume that empty values in the ReturnDate column represent books that have yet to be returned by their borrowers.

Schema Design

The dataset, as it was described in the previous section, is perfectly well suited to providing information about borrows at a particular point in time. However, it would be a very poor way of storing data long-term, especially given how often books are borrowed and returned and how much duplicate data would be stored if, for example, the same book were to be checked out multiple times. In other words, this dataset is not well normalized. Data normalization refers to the removal of redundancy and the enforcement of data integrity in one's dataset or database design (Richman, 2023). The primary goal of this project is to take our library borrows dataset and design a new, normalized data schema. That schema will then be implemented into a PostgreSQL database. Lastly, I will illustrate how the database can be used with various queries and how the new database provides the opportunity for query optimization with database indexes.

The first step towards normalizing this database is to identify all of the “entities” having information stored about them. Entities are distinct objects that are being described by your data (Belcic & Stryker, n.d.). Every column in a dataset is describing one of these entities. A normalized dataset will generally have each table representing one of these entities. Once you have identified these entities, you can create an Entity-Relationship Diagram which describes how these entities are related to each other in your dataset (Belcic & Stryker, n.d.).

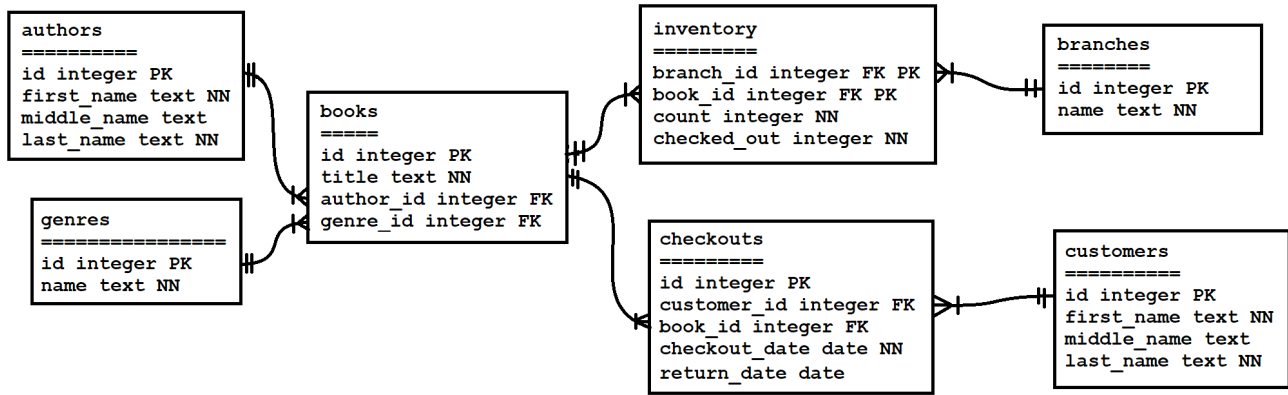
The following diagram shows my first attempt at mapping out the entities described in this dataset.



This diagram was mostly conceptual—it includes several fields that are not actually included in the data such as edition and description for the books, and it also includes fewer entities than I eventually identified in the final schema. Still, creating this diagram allowed me to begin to conceptualize how all of the data stored in the dataset related to each other.

Each record represented a particular instance of a borrow, here described as a “checkout.” Each checkout must be coming from a particular library branch, must have a single borrower attached to it, and a book being checked out. Note that the author and genre of each book is currently being treated as descriptors of the book entity when they could also be considered entities on their own. Also note the inclusion of an inventory entity; this represents the inventory of books available at each library branch. While the dataset does not provide any information about books in the inventory that are not being borrowed currently, I still chose to include it. I did so because to design a database that could actually be used by a library system, such a table would absolutely need to be present. And, while the dataset would not give us any information about books in the inventory that haven’t been checked out, it does tell us the branch associated with check out, so at least those books can be included in the inventory.

With these notes considered, I drafted several new entity-relationship diagrams until I settled on the following.



Note that this diagram does not use the flowchart style shown in the previous one. This is simply a different type of diagram using a form of notation called crow's foot notation (Belcic & Stryker, n.d.). It is meant to more clearly show how each entity exists in its own table within the database. Both the flowchart style and this more explicit database style are common ways of creating entity-relationship diagrams (Belcic & Stryker, n.d.). This type of diagram not only shows which entities are related to each other, but precisely how they will be related to each other within the database.

The marks on the lines connecting the entities describe the relationship in terms of one-to-one, one-to-many, or many-to-many. This data schema does not contain any instances of a one-to-one relationship. An example of a one-to-many relationship in this dataset is that of the genres and books table. Each book is only going to be classified into one genre for organizational purposes, but each genre will contain many different books. This relationship is indicated by the two swipes on the line next to the genres table and the single swipe followed by a three-fingered branch next to the books table. Those symbols are used to indicate a one-to-many relationship, in this case that one genre has many books associated with it.

An example of a many-to-many relationship in the diagram is that of the books and branches tables. The same book might be present in multiple different library branches, and the same branch will have many different books. Describing many-to-many relationships without including lots of data duplication generally requires the use of junction tables (Lu, 2026). Each record in a junction table uniquely identifies a combination of objects from each table. In the case of this entity-relationship diagram, the inventory table serves as a sort of junction table between the books and branches inventory. Each record represents one branch's inventory of one book and stores how many copies that branch has of that book. In this way, a many-to-many relationship can be broken down into two many-to-one relationships.

The last thing that should be noted about this entity-relationship diagram is that each table has a unique identifier for each record called a primary key. All of the columns that will be included in a table are listed below the table name, and the primary key column is denoted with a "PK" next to the column name. Whenever a table wants to reference a value from another entity, it simply uses the unique identifier of that value. This way, no additional information about the entities needs to be duplicated across multiple tables. A column which

includes references to another table's primary key is called a foreign key column, denoted with "FK." Note that the relationships in the diagrams do not only connect the tables together, but specifically connect the relevant primary key-foreign key relationships between each table. Lastly, note that junction tables will have two different primary key columns, which also happen to be the foreign key columns referencing the two entities being tied together by this junction. This is generally preferred to creating a unique identifier for each relationship in the junction as each record should uniquely represent one pair of objects from the two tables. As such, the two columns can be used together to uniquely identify each row in the junction table.

Data normalization is measured in terms of "normal forms," criteria for normalization which progressively build on one another to create stricter and stricter measures of normalization (Richman, 2023). The most basic is the first normal form, which requires a schema to ensure that cells in a table never include lists or arrays and that a column represents an atomic value, meaning that it cannot be divided into multiple different columns (Richman, 2023). My library schema adheres to these requirements; an example of the atomic requirement is that the first and last names have been split into separate columns when they were originally written as one single column. In other words, they were not atomic originally, and now they are.

The second normal form builds on the first, adding the stipulation that any column in a table must depend on an entire primary key, not just part of it (Upadhyay, 2025). What this means, is if there is a column with a composite primary key (such as the inventory table in my schema) all columns must be specific to each individual combination. If, in my schema, I had branch names stored in the inventory column, this would violate this rule as the branch name is not dependent on the books in the inventory. My schema adheres to this rule by keeping branch-specific columns in the separate branches table so that the inventory table's columns depend solely on the composite primary key of both the branch ID and the book ID.

The third normal form continues building on the previous two, including all their requirements and adding another: the schema must include no transitive dependencies (Richman, 2023). This means that columns must depend solely on the primary key and not on any other non-primary key. There is no obvious example of such a transitive dependency in the original schema, and it can be easily verified that no such dependencies exist in the final schema either. While there are more strict normal forms such as the Boyce-Codd normal form, the fourth normal form, and the fifth normal form, it is common to use the third normal form as a target when designing a database as it ensures a sufficient amount of data integrity without becoming so strict that data gets spread over too many columns (Crudu, 2025). In other words, it provides a sufficient balance between integrity and efficiency.

Importing Data

With the new schema defined, the next step would be to create a new database that follows it and import the library data. I used Docker Compose to set up and launch the PostgreSQL server. The file "docker-compose.yml" which is accessible from the root directory of the GitHub repository, defines the docker image and image settings to use and defines a volume so that the data in the server will persist even if the container is shut down. I can then

easily launch the server by running the command “docker compose up” in that directory assuming that the Docker engine is already running (Docker, Inc., n.d.). I then used the PSQL shell program on my machine to connect to the server to run the scripts associated with importing the data (and later querying it and optimizing the queries).

In order to import the data into the database I first needed to define a dummy table with the same format as the original CSV file. I would then be able to query this dataset using SQL commands to move the data into the new tables. The script “setup.sql,” located on the root directory is responsible for this.

```
CREATE TABLE temp_raw_data (  
    BookID integer,  
    Title text,  
    Author text,  
    Genre text,  
    BranchID integer,  
    BranchName text,  
    BorrowerID integer,  
    BorrowerName text,  
    BorrowDate text,  
    ReturnDate text  
);  
  
\copy temp_raw_data FROM 'dbsetup/city_library_data.csv' CSV HEADER;
```

Once the data was copied into the new table, then the tables for each entity in the new schema needed to be created and then filled with data. Each table’s definition and import was moved into a separate .sql file inside of the dbsetup directory. The script setup.sql then calls each in sequence.

```
\i dbsetup/import_author.sql  
\i dbsetup/import_genre.sql  
\i dbsetup/import_book.sql  
\i dbsetup/import_branch.sql  
\i dbsetup/import_inventory.sql  
\i dbsetup/import_member.sql  
\i dbsetup/import_checkout.sql
```

I will now go through each of the import scripts, explaining how I manipulated the data to ensure it could fit inside the new schema. First, I will cover the “import_author.sql” script, responsible for creating the book authors table and populating it. Inside of this (and many of the import scripts) are commented out SQL queries used to glean important information about the dataset. For example, in this script can be found commented out queries which used

regular expressions (regex) to identify any names that did not follow a “Firstname Lastname” format.

```
/* Lists authors whose listed names do not follow a "Firstname Lastname" format.
SELECT bookID, author
FROM temp_raw_data
WHERE author !~ '^[A-Za-z]+ [A-Za-z]+$';
*/
```

Running this query reveals a number of authors who have prefixes like “Mr.,” “Mrs.,” or “Dr.” and suffixes such as “Jr.,” “III,” or “DDS.”

bookid	author
20	Mr. Roger Hardin
48	Mr. John Heath
49	Dr. Carol Garcia
65	Mark Flores DDS
77	Seth Bishop Jr.
96	Danielle Sanders DDS
98	Bryan Jacobs III

To prepare these names for the new schema, “import_author.sql” first runs a command to strip the prefixes off of these names.

```
/* Strips prefixes from the author field wherever present. */
UPDATE temp_raw_data
SET author = ARRAY_TO_STRING((STRING_TO_ARRAY(author, ' ')[2:]), ' ')
WHERE author ~ '^(Mr\.|Mrs\.|Dr\.) [A-Za-z]+ [A-Za-z]+$';
```

Then another command is run to strip the suffixes.

```
/* Removes suffixes from all authors whose names do not follow a "Firstname Lastname" format. */
UPDATE temp_raw_data
SET author = ARRAY_TO_STRING((STRING_TO_ARRAY(author, ' ')[1:2]), ' ')
WHERE author !~ '^[A-Za-z]+ [A-Za-z]+$';
```

Then, the new authors table is created according to the schema, and the names are imported using the PostgreSQL function `STRING_TO_ARRAY()` to split the names into separate first and last names.

```

/* Creates new authors table */
CREATE TABLE authors (
  id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  first_name text NOT NULL,
  last_name text NOT NULL
);

/* Inserts first and last names into new authors table */
INSERT INTO authors (first_name, last_name)
SELECT DISTINCT
  ARRAY_TO_STRING((STRING_TO_ARRAY(author, ' ')[1:1], ' ') AS first,
  ARRAY_TO_STRING((STRING_TO_ARRAY(author, ' ')[2:2], ' ') AS last
FROM temp_raw_data;

```

The next script to be called is the “import_genre.sql” script, which is notably simpler. It only needs to define the new schema, select all the distinct genres present in the temporary data, and insert them into the new table.

```

/* Create a genres table. */
CREATE TABLE genres (
  id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name text NOT NULL
);

/* Import the genres from the temp table into the genres table. */
INSERT INTO genres (name)
SELECT DISTINCT genre
FROM temp_raw_data;

```

The next script is “import_book.sql,” which naturally must refer to the authors and genres tables. The command used to import this data is somewhat complex as it has to join the new authors and genres table to the temporary table in order to get a reference to each book’s genre and author IDs. The latter has to be accomplished by once again splitting the author name in the raw data table using STRING_TO_ARRAY(). Once done, however, it is fairly simple to import the data into the new schema. We simply select the book’s title along with the author ID and genre ID identified.


```

/* Creates new books table. */
CREATE TABLE books (
  id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  title TEXT NOT NULL,
  author_id INTEGER DEFAULT 0,
  genre_id INTEGER DEFAULT 0,
  CONSTRAINT fk_author
    FOREIGN KEY (author_id)
    REFERENCES authors(id)
    ON DELETE SET DEFAULT,
  CONSTRAINT fk_genre
    FOREIGN KEY (genre_id)
    REFERENCES genres(id)
    ON DELETE SET DEFAULT
);

/* Inserts book information from temp data into books table, using foreign keys to connect to author and genre tables. */
INSERT INTO books (id, title, author_id, genre_id)
SELECT t.bookID, t.title, a.id, g.id
FROM temp_raw_data t
LEFT JOIN authors a
ON
  a.first_name = (STRING_TO_ARRAY(t.author, ' ')[1] AND
  a.last_name = (STRING_TO_ARRAY(t.author, ' ')[2]
LEFT JOIN genres g
ON t.genre = g.name;

```

The next script to be run is “import_branch.sql,” which is another fairly simple one. It simply defines the structure of the branches table then imports the distinct branch names and their associated IDs.

```

/* Create new table for branches. */
CREATE TABLE branches (
  id integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name text NOT NULL
);

/* Insert data from temp data into new branches table. */
INSERT INTO branches (id, name)
SELECT DISTINCT branchid, branchname
FROM temp_raw_data
ORDER BY branchid;

```

The next script is the “import_inventory.sql” script, which despite being a junction table, is in fact quite simple.

```

/* Create table representing the library system's inventory.
Each row represents a particular branch's inventory for a particular book. */
CREATE TABLE inventory (
    branch_id integer,
    book_id integer,
    count integer NOT NULL DEFAULT 0,
    CONSTRAINT pk_inventory
        PRIMARY KEY (branch_id, book_id),
    CONSTRAINT fk_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(id)
        ON DELETE CASCADE,
    CONSTRAINT fk_book
        FOREIGN KEY (book_id)
        REFERENCES books(id)
        ON DELETE CASCADE
);

/* Insert data into inventory tables. */
INSERT INTO inventory (branch_id, book_id, count)
SELECT branchid, bookid, COUNT(*)
FROM temp_raw_data
GROUP BY bookid, branchid;

```

Next is the “import_member.sql” script, which similarly to the authors script has to strip the prefixes off of the borrower names before importing them.

```

/* Remove prefixes from borrower names who have them. */
UPDATE temp_raw_data
SET borrowername = ARRAY_TO_STRING((STRING_TO_ARRAY(borrowername, ' ')[2:], ' '))
WHERE borrowername ~ '^(Mr\.)|(Mrs\.)|(Dr\.)) [A-Za-z]+ [A-Za-z]+';

/* Remove suffixes from borrower names who have them. */
UPDATE temp_raw_data
SET borrowername = ARRAY_TO_STRING((STRING_TO_ARRAY(borrowername, ' ')[1:2], ' '))
WHERE borrowername !~ '^[A-Za-z]+ [A-Za-z]+$';

```

```

/* Create new members table. */
CREATE TABLE members (
    id integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name text NOT NULL,
    last_name text NOT NULL
);

/* Import members into new tables. */
INSERT INTO members (first_name, last_name)
SELECT DISTINCT
    ARRAY_TO_STRING((STRING_TO_ARRAY(borrowername, ' ')[1:1], ' ') AS first,
    ARRAY_TO_STRING((STRING_TO_ARRAY(borrowername, ' ')[2:2], ' ') AS last
FROM temp_raw_data;

```

The final script in the import process is “import_checkout.sql,” which despite defining a fairly complex table, has a fairly simple import command.

```

/* Create table representing individual book rentals. */
CREATE TABLE checkouts (
    id integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    member_id integer NOT NULL,
    book_id integer NOT NULL,
    branch_id integer NOT NULL,
    checkout_date date NOT NULL,
    return_date date,
    CONSTRAINT fk_members
        FOREIGN KEY (member_id)
        REFERENCES members(id)
        ON DELETE CASCADE,
    CONSTRAINT fk_book
        FOREIGN KEY (book_id)
        REFERENCES books(id)
        ON DELETE CASCADE,
    CONSTRAINT fk_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(id)
        ON DELETE CASCADE
);

```

```

INSERT INTO checkouts (member_id, book_id, branch_id, checkout_date, return_date)
SELECT
    m.id AS member_id,
    t.bookID AS book_id,
    t.branchID AS branch_id,
    TO_DATE(borrowdate, 'YYYY-MM-DD') AS checkout_date,
    TO_DATE(returndate, 'YYYY-MM-DD') AS return_date
FROM temp_raw_data t
JOIN members m
ON
    m.first_name = ARRAY_TO_STRING((STRING_TO_ARRAY(borrowername, ' ')[1:1], ' ') AND
    m.last_name = ARRAY_TO_STRING((STRING_TO_ARRAY(borrowername, ' ')[2:2], ' '));

```

Once each of the import scripts have been run, “setup.sql” then drops the temporary table used to hold the raw data.

```

23
24 DROP TABLE temp_raw_data;

```

The entire database setup process can therefore be done just by running the script in PSQL using the command “\i setup.sql.”

Querying the Database

To illustrate how this new schema can be used to answer various organizational problems, I have come up with three questions of the kind you might often see in a library management context. The first question is, “Is the book ‘Million size country.’ currently available anywhere in the library network?” One could imagine this question being relevant if a library member wanted to look for a book and was willing to drive to any of the library’s branches to find it.

The query written to answer this question (and the other two that will appear later) is contained in the script “queries.sql” inside the “querying” directory. The following query is used to find the answer to this question in the database. Note the use of common table expressions (CTEs) to combine the search for available copies of the book and total copies of the book into a single query.

```

WITH total AS (
  SELECT count as total_copies
  FROM inventory i
  JOIN books b ON i.book_id = b.id
  WHERE title = 'Million size country.'
),
available AS (
  SELECT COUNT(*) as available_copies
  FROM checkouts c
  JOIN books b on c.book_id = b.id
  WHERE b.title = 'Million size country.' AND return_date IS NOT NULL
)
SELECT a.available_copies, t.total_copies
FROM total t
CROSS JOIN available a;

```

This query yields the following output.

available_copies	total_copies
0	1

(1 row)

As such, the answer to this first question is no. While there is a copy in the library network generally, there is no copy that is currently available.

The second question is “Which branch has the largest inventory of history books?” One could imagine this question being relevant if a library member wanted to conduct some history research in-person. Such a member would want to identify which branch gives them the best chance of finding as many books as they need to complete their research. The query used to answer this question and the resulting output are as follows.

```

SELECT br.name branch_name, SUM(count) as history_book_inventory
FROM inventory i
JOIN books bk ON bk.id = i.book_id
JOIN genres g ON g.id = bk.genre_id
JOIN branches br ON br.id = i.branch_id
WHERE g.name = 'History'
GROUP BY br.name
ORDER BY history_book_inventory DESC;

```

branch_name	history_book_inventory
Branch-3	17
Branch-2	15
Branch-5	11
Branch-1	10
Branch-4	8
(5 rows)	

Note that this query lists the branches in order of the size of their history inventory and identifies Branch-3 as the most ideal location with 17 history books.

The third and final question is, “Which branch has the smallest percentage of their books checked out?” One could imagine a library system administrator wanting to gauge the general popularity of each individual branch with such a question. The query used to answer it is shown below, along with its output.

```
WITH inventory_per_branch AS (
    SELECT branch_id, COUNT(*) AS count
    FROM inventory
    GROUP BY branch_id
),
checked_out_per_branch AS (
    SELECT branch_id, COUNT(*) AS count
    FROM checkouts
    WHERE return_date IS NULL
    GROUP BY branch_id
)

SELECT
    b.id,
    b.name AS branch,
    k.count AS checked_out,
    i.count AS total,
    ROUND(100.0 * k.count / i.count, 2) AS percentage
FROM inventory_per_branch i
JOIN checked_out_per_branch k
ON i.branch_id = k.branch_id
JOIN branches b
ON i.branch_id = b.id
ORDER BY percentage;
```

id	branch	checked_out	total	percentage
2	Branch-2	49	106	46.23
5	Branch-5	38	82	46.34
1	Branch-1	48	99	48.48
4	Branch-4	51	105	48.57
3	Branch-3	61	108	56.48

(5 rows)

As can be seen, the branch with the smallest proportion of books currently checked out is Branch-2 at about 46%, but Branch-5 is very close behind.

Optimizing Queries

An essential skill for anybody managing a large database is query optimization through indexing. A database index is a data structure which provides a fast way to identify certain rows in the table based on the values of a particular column (PostgreSQL Global Development Group, n.d.). An example in this database of a query one might want to optimize with an index is that of a librarian seeing which books are currently checked out from the library. With the current schema, a book is considered checked out if the return_date column in the checkouts table is null. As such, placing an index on that column would allow the database to quickly identify rows where the value is null, speeding up this query.

Now, in this project, none of the tables hold a huge amount of data which limits the ability of indexes to speed-up queries. This is because the query planner will often default towards scanning an entire table, even if there is an index, if the table isn't that long in the first place. As such, there is less opportunity to show off indexing with this table than there would be if the source dataset was larger. However, I still did optimize one query significantly, and that is the first query listed in "queries.sql."

This query was meant to find if the book "Million size country." is currently available anywhere in the library network. By using the SQL keywords "EXPLAIN ANALYZE" before the query, PSQL will print out the query plan used to answer the query, along with a runtime. It provides the following output.

```

librarydb=# FROM total c
librarydb=# CROSS JOIN available a;

QUERY PLAN

-----
Nested Loop (cost=32.20..41.55 rows=1 width=12) (actual time=1.556..1.561 rows=1.00 loops=1)
 Buffers: shared hit=7 read=10
-> Hash Join (cost=11.26..20.59 rows=1 width=4) (actual time=0.915..0.920 rows=1.00 loops=1)
   Hash Cond: (i.book_id = b.id)
   Buffers: shared hit=2 read=6
   -> Seq Scan on inventory i (cost=0.00..8.00 rows=500 width=8) (actual time=0.252..0.282 rows=500.00 loops=1)
     Buffers: shared read=3
   -> Hash (cost=11.25..11.25 rows=1 width=4) (actual time=0.583..0.583 rows=1.00 loops=1)
     Buckets: 1024 Batches: 1 Memory Usage: 9kB
     Buffers: shared hit=2 read=3
     -> Seq Scan on books b (cost=0.00..11.25 rows=1 width=4) (actual time=0.032..0.057 rows=1.00 loops=1)
       Filter: (title = 'Million size country.'::text)
       Rows Removed by Filter: 499
       Buffers: shared hit=2 read=3
-> Aggregate (cost=20.94..20.95 rows=1 width=8) (actual time=0.638..0.639 rows=1.00 loops=1)
   Buffers: shared hit=5 read=4
   -> Hash Join (cost=11.26..20.94 rows=1 width=0) (actual time=0.636..0.637 rows=0.00 loops=1)
     Hash Cond: (c.book_id = b_1.id)
     Buffers: shared hit=5 read=4
     -> Seq Scan on checkouts c (cost=0.00..9.00 rows=253 width=4) (actual time=0.232..0.562 rows=253.00 loops=1)
       Filter: (return_date IS NOT NULL)
       Rows Removed by Filter: 247
       Buffers: shared read=4
     -> Hash (cost=11.25..11.25 rows=1 width=4) (actual time=0.039..0.039 rows=1.00 loops=1)
       Buckets: 1024 Batches: 1 Memory Usage: 9kB
       Buffers: shared hit=5
       -> Seq Scan on books b_1 (cost=0.00..11.25 rows=1 width=4) (actual time=0.008..0.036 rows=1.00 loops=1)
         Filter: (title = 'Million size country.'::text)
         Rows Removed by Filter: 499
         Buffers: shared hit=5
Planning:
 Buffers: shared hit=213 read=46
Planning Time: 1.227 ms
Execution Time: 1.678 ms
(34 rows)

```

In the query plan, the primary targets of optimization will be the sequential scans, since these are steps in the process where an entire table has to be scanned. Wherever there is a sequential scan that we know to be related to a filtering operation, an index should be able to optimize it. For this query plan, there are four sequential scans. First, on the inventory column which will be filtering out the books that don't include certain IDs. Second, on the books column will be filtering out books based on title. Third on the the checkouts table, once again when filtering by book ID. The fourth and final one is a second scan on books, being repeated because it exists in a separate subquery from the previous one due to the use of CTEs.

Therefore, to optimize the query, I created the following three indexes.

```

CREATE INDEX idx__books__title ON books(title);
CREATE INDEX idx__inventory__book_id ON inventory(book_id);
CREATE INDEX idx__checkouts__book_id ON checkouts(book_id);

```

Now, with these indexes, the query plan is altered to be significantly more efficient.


```

QUERY PLAN
-----
Nested Loop (cost=17.14..33.20 rows=1 width=12) (actual time=0.303..0.320 rows=1.00 loops=1)
  Buffers: shared hit=10 read=2
  -> Nested Loop (cost=0.55..16.59 rows=1 width=4) (actual time=0.202..0.218 rows=1.00 loops=1)
    Buffers: shared hit=4 read=2
    -> Index Scan using idx_books_title on books b (cost=0.27..8.29 rows=1 width=4) (actual time=0.092..0.107 rows=1.00 loops=1)
      Index Cond: (title = 'Million size country.'::text)
      Index Searches: 1
      Buffers: shared hit=1 read=2
    -> Index Scan using idx_inventory_book_id on inventory i (cost=0.27..8.29 rows=1 width=8) (actual time=0.023..0.023 rows=1.00 loops=1)
      Index Cond: (book_id = b.id)
      Index Searches: 1
      Buffers: shared hit=3
  -> Aggregate (cost=16.59..16.60 rows=1 width=8) (actual time=0.071..0.072 rows=1.00 loops=1)
    Buffers: shared hit=6
    -> Nested Loop (cost=0.55..16.59 rows=1 width=0) (actual time=0.056..0.056 rows=0.00 loops=1)
      Buffers: shared hit=6
      -> Index Scan using idx_books_title on books b_1 (cost=0.27..8.29 rows=1 width=4) (actual time=0.024..0.024 rows=1.00 loops=1)
        Index Cond: (title = 'Million size country.'::text)
        Index Searches: 1
        Buffers: shared hit=3
      -> Index Scan using idx_checkouts_book_id on checkouts c (cost=0.27..8.29 rows=1 width=4) (actual time=0.030..0.030 rows=0.00 loops=1)
        Index Cond: (book_id = b_1.id)
        Filter: (return_date IS NOT NULL)
        Rows Removed by Filter: 1
        Index Searches: 1
        Buffers: shared hit=3
Planning:
  Buffers: shared hit=81 read=9
Planning Time: 2.888 ms
Execution Time: 0.813 ms
(30 rows)

```

Note that all sequential scans have been replaced with index scans, and the execution time has been reduced from 1.6 milliseconds to 0.8 milliseconds. This constitutes a significant improvement in efficiency for this query. The difference in efficiency between the non-optimized and optimized versions of the query would also prove to grow wider as the number of elements in the database increases, but even with this fairly small dataset, the improvement is clear.

Next Steps

This project has illustrated the general process of designing and implementing a normalized database using a poorly normalized dataset. It also illustrated how one can query the dataset using intermediate PostgreSQL querying techniques and optimize commonly performed queries using indexes.

One of the main ways this project could be extended is to increase the number of elements in the original dataset, and then add more queries to be optimized. As mentioned in the Optimizing Queries section, with a dataset as small as the one used here, the query planner will often default to performing sequential scans even if an index is present. Increasing the number of elements in the dataset would allow for indexes to be more effective in optimizing, since the query planner would be more likely to choose index scans over sequential ones.

One major next step for the project would be to modify and prepare it for use in a real-world library setting. Several changes would need to be made for this to be prepared. The checkouts table would likely need to be restructured to include due dates and possibly to alter the way that return dates are stored so there are fewer or no null values. The relationship between the books and authors table would need to be updated from a one-to-many relationship to a many-to-many relationship since books can have multiple authors. Many

additional fields would need to be added to various tables: for example, descriptions would need to be added to the books table and addresses to the members table. It could also be expanded to include information about employees of the library as well. There are many different ways the database would need to be expanded significantly to account for all of the organizational needs of an actual library system.

The final next step I will mention here would be the creation of a library catalog. This would be a program that provides a user interface that laypeople can use to interact with the database. With the database as it appears now, this would look like a program which can search the inventory table for relevant books based on title or author search parameters provided by the user. The program would be responsible for turning these parameters into SQL queries that can be used to retrieve the data.

References

Belcic, I. & Stryker, C. (n.d.). *What is an entity relationship diagram?* IBM. <https://www.ibm.com/think/topics/entity-relationship-diagram>

Crudu, V. (2025, July 27). A Beginner's Guide to Database Normalization – Key Concepts and Techniques Explained. MoldStud. <https://moldstud.com/articles/p-a-beginners-guide-to-database-normalization-key-concepts-and-techniques-explained>

Docker, Inc. (n.d.). *Docker documentation*. <https://docs.docker.com/>

Faker contributors. (n.d.). *Faker documentation*. Faker. <https://faker.readthedocs.io/>

Lu, T. (2026, February 3). *Many to Many Relationship: A Guide to Database Design*. Datacamp. <https://www.datacamp.com/blog/many-to-many-relationship>

PostgreSQL Global Development Group. (n.d.). *Indexes*. In PostgreSQL 18 documentation. <https://www.postgresql.org/docs/current/indexes.html>

Richman, J. (2023, July 30). *Data Normalization: Types, Techniques & Examples*. Estuary. <https://estuary.dev/blog/data-normalization/>

Upadhyay, M. (2025, July 12). *Second Normal Form (2NF)*. GeeksForGeeks. <https://www.geeksforgeeks.org/dbms/second-normal-form-2nf/>